Score-Based Generative Models, Denoising Score Matching & Denoising Diffusion Probabilistic Models

Abstract. This report explains the ideas behind diffusion-based image generation, starting from the basic concept of a score function, progressing through Denoising Score Matching (DSM), and arriving at the Denoising Diffusion Probabilistic Model (DDPM) of Ho et al. (2020). Each mathematical concept is explained in plain language before the formula is introduced. Diagrams are provided throughout to build visual intuition.

1. Score Functions and Score-Based Generative Models

1.1 What Problem Are We Solving?

A generative model must learn a distribution over images and draw new samples from it. Classical approaches (GANs, VAEs) model the distribution *indirectly*. Score-based models take a different route: instead of learning the probability $p(\mathbf{x})$ itself, they learn its *gradient with respect to the image pixels*. This gradient is called the **score function**.

Definition: Score Function

Given any distribution $p(\mathbf{x})$ over images $\mathbf{x} \in \mathbb{R}^d$, the **score function** is

$$s(\mathbf{x}) \;=\; \nabla_{\mathbf{x}} \log p(\mathbf{x}).$$

It is a vector field—at every point in image space, it points in the direction of increasing probability (i.e., toward more realistic images).

Why not just model $p(\mathbf{x})$? To write down $p(\mathbf{x})$ as a proper probability distribution, you need a normalizing constant $Z = \int e^{f(\mathbf{x})}\, d\mathbf{x}$, which is intractable in high dimensions. The score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ cancels Z out entirely, making it much easier to work with.

1.2 Visualizing the Score Field

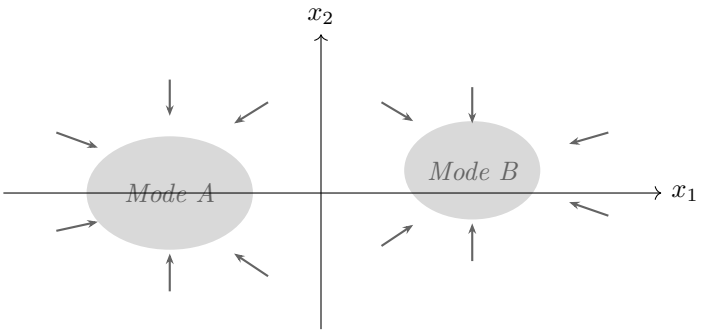

Figure 1: Score field arrows point toward high-density regions (the two data modes).

The score vector field $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ for a two-mode distribution. Arrows show the direction of steepest ascent of the log-density. Starting from any point, following the arrows leads to a data mode.

1.3 Sampling with Langevin Dynamics

Once we have the score, we can generate new samples by starting from random noise and iteratively nudging the point in the direction the score points, with a small amount of randomness added at each step to ensure full exploration of the distribution. This is called **Langevin dynamics**:

Langevin MCMC Update Rule

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\delta}{2} \underbrace{s_\theta(\mathbf{x}_k)}_{\text{push toward data}} + \underbrace{\sqrt{\delta}\,\mathbf{z}_k}_{\text{random exploration}}, \qquad \mathbf{z}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

With small step size δ and many steps $K \to \infty$, samples converge to $p(\mathbf{x})$ without ever computing Z.

1.4 Why Naive Score Matching Fails

The most direct way to learn $s_\theta(\mathbf{x})$ is *explicit score matching*: minimise $\mathbb{E}[\frac{1}{2}\|s_\theta(\mathbf{x})\|^2 + \mathrm{tr}(\nabla_{\mathbf{x}} s_\theta(\mathbf{x}))]$. The trace-of-Jacobian term costs $O(d^2)$ to evaluate—completely infeasible for a 256×256 image ($d \approx 200{,}000$). Moreover, real images live on a low-dimensional manifold inside the high-dimensional pixel space, so most of that space has essentially zero data density and the score is meaningless there.

2. Denoising Score Matching (DSM)

2.1 The Core Idea: Add Noise, Then Learn to Remove It

Vincent (2011) discovered an elegant fix: instead of estimating the score of the clean data distribution, *perturb the data with Gaussian noise* and estimate the score of the resulting noisy distribution. Because we control the noise, the target score has a closed form—no Jacobian needed.

How it works. Take a clean image $\mathbf{x}_0$. Add Gaussian noise: $\tilde{\mathbf{x}} = \mathbf{x}_0 + \sigma\,\mathbf{z}$, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The noisy distribution is $p(\tilde{\mathbf{x}} \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, \sigma^2\mathbf{I})$, whose score is:

$$\nabla_{\tilde{\mathbf{x}}} \log p(\tilde{\mathbf{x}} \mid \mathbf{x}_0) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}_0}{\sigma^2} = -\frac{\mathbf{z}}{\sigma}.$$

This is just the *negative scaled noise*: the direction from the noisy image back to the clean one. We train a network $s_\theta(\tilde{\mathbf{x}}, \sigma)$ to predict this:

DSM Objective (Vincent, 2011)

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_\sigma\, \mathbb{E}_{\mathbf{x}_0}\, \mathbb{E}_{\tilde{\mathbf{x}} \sim \mathcal{N}(\mathbf{x}_0, \sigma^2\mathbf{I})} \left[\left\| s_\theta(\tilde{\mathbf{x}}, \sigma) + \tfrac{\tilde{\mathbf{x}} - \mathbf{x}_0}{\sigma^2} \right\|^2 \right].$$

Minimising this is equivalent to training a *denoising autoencoder* that predicts the clean image from the noisy one.

2.2 Multi-Scale Noise and Annealed Langevin Sampling

A single noise level σ is not enough. Song & Ermon (2019) proposed using a *geometric ladder* of T noise levels $\sigma_1 < \sigma_2 < \cdots < \sigma_T$:

- **Large** σ_T (heavy noise): the data fills all of space, so Langevin sampling can explore freely and find different modes.
- **Small** σ_1 (light noise): the score closely approximates the true data score; fine details are refined.

A single network $s_\theta(\mathbf{x}, \sigma_i)$ conditioned on σ_i is trained at all levels jointly. Sampling then runs Langevin from σ_T down to σ_1 in sequence. This is **annealed Langevin dynamics**, and it is conceptually identical to the reverse process in DDPM.

3. DDPM: Ho et al. (2020)

3.1 The Big Picture

DDPM formalises the noise ladder from DSM into a clean probabilistic framework. There are two processes:

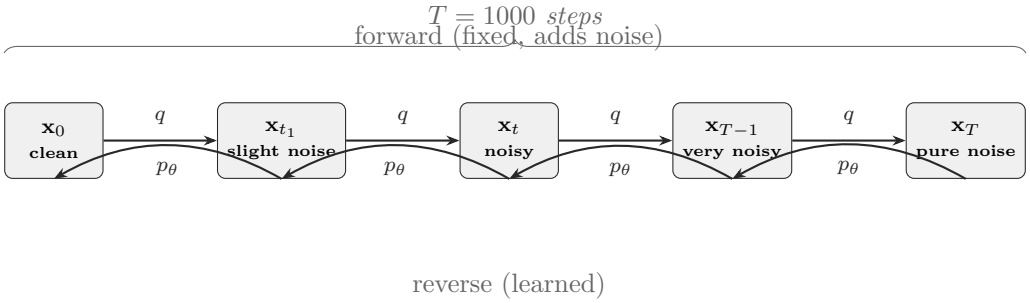

Figure 2: The DDPM framework. The forward process (top arrows) gradually destroys an image over T steps by adding Gaussian noise. The reverse process (bottom arrows) is learned by a neural network and reconstructs an image from pure noise.

3.2 Forward Process: Gradually Adding Noise

The forward process is a fixed (not learned) Markov chain. At each step t, a tiny amount of Gaussian noise is mixed in:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \;=\; \mathcal{N}\Big(\mathbf{x}_t;\; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\; \beta_t \mathbf{I}\Big)\,.$$

Here β_t is a small number (the *noise schedule*) that grows from $\approx 10^{-4}$ at $t = 1$ to ≈ 0.02 at $t = T$. The $\sqrt{1-\beta_t}$ factor slightly shrinks the signal so that the overall variance stays controlled.

One-step shortcut. Remarkably, we can jump directly from $\mathbf{x}_0$ to any noisy $\mathbf{x}_t$ without simulating all t steps. Defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ (the cumulative signal retention):

Closed-form Marginal (the "Jump Formula")

$$\mathbf{x}_t \;=\; \sqrt{\bar{\alpha}_t}\;\underbrace{\mathbf{x}_0}_{\text{clean image}} \;+\; \sqrt{1-\bar{\alpha}_t}\;\underbrace{\boldsymbol{\varepsilon}}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\,, \qquad \bar{\alpha}_t \xrightarrow{t\to T} 0.$$

At $t = T$, essentially all signal is gone and $\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.

3.3 Reverse Process: Learned Denoising

The reverse process runs backwards: starting from pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, a neural network $\boldsymbol{\varepsilon}_\theta$ iteratively removes noise. Each reverse step is also Gaussian:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \;=\; \mathcal{N}\big(\mathbf{x}_{t-1};\; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\; \sigma_t^2 \mathbf{I}\big)\,.$$

The mean $\boldsymbol{\mu}_\theta$ is expressed in terms of the *predicted noise* $\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)$:

Reverse Step Mean

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) \;=\; \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t \;-\; \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\right).$$

Intuitively: take the current noisy image, subtract the predicted noise (scaled appropriately), and divide to undo the signal shrinkage.

Connection to score matching. Since $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) = -\boldsymbol{\varepsilon}/\sqrt{1-\bar{\alpha}_t}$, the noise predictor $\boldsymbol{\varepsilon}_\theta$ is simply a re-scaled score estimate: $s_\theta(\mathbf{x}_t, t) = -\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)/\sqrt{1-\bar{\alpha}_t}$. **DDPM's reverse process is DSM in disguise.**

3.4 Simplified Training Loss

The theoretically correct objective (the ELBO) is complicated. Ho et al. found that simply training the network to predict the noise $\boldsymbol{\varepsilon}$ with a plain mean-squared-error loss works better in practice:

Simplified Training Loss — Ho et al. (2020), Eq. 14

$$\mathcal{L}_{\text{simple}} \;=\; \mathbb{E}_{t,\,\mathbf{x}_0,\,\boldsymbol{\varepsilon}}\Big[\big\|\boldsymbol{\varepsilon} \;-\; \boldsymbol{\varepsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon},\; t\big)\big\|^2\Big].$$

In words: sample a random timestep t, add the corresponding amount of noise to a real image to get $\mathbf{x}_t$, and ask the network to predict which noise was added.

The full training and sampling loops are summarized in Table 1.

Table 1: DDPM Training and Sampling Algorithms (Ho et al., 2020)

Training (repeat until convergence)	**Sampling (to generate one image)**
1. Sample a clean image $\mathbf{x}_0$ from the dataset.	1. Sample pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Pick a random timestep $t \in \{1, \ldots, T\}$.	2. For $t = T$ down to 1:
3. Sample noise $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and form $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}$.	a. Predict noise: $\hat{\boldsymbol{\varepsilon}} = \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)$.
4. Compute loss $\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\|^2$ and update the network.	b. Compute mean $\boldsymbol{\mu}_\theta$ and sample $\mathbf{x}_{t-1}$.
	3. Return $\mathbf{x}_0$ as the generated image.

4. U-Net: The Neural Network Inside DDPM

4.1 Why U-Net?

The noise predictor $\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)$ receives a noisy image and must output a noise map of the *same spatial size*. It also needs to understand both fine pixel-level details and large structural patterns simultaneously. The **U-Net**, originally developed for biomedical image segmentation, is a perfect fit.

4.2 Key Components

Residual Blocks (ResBlocks). Each ResBlock applies two convolutions with GroupNorm and SiLU activation, plus a skip connection. The *timestep* t is embedded via sinusoidal encoding (identical to Transformers) and injected additively:

$$\mathbf{h} \;\leftarrow\; \text{Conv}(\text{SiLU}(\text{GN}(\mathbf{h}))) + \text{Linear}(\mathbf{e}_t),$$

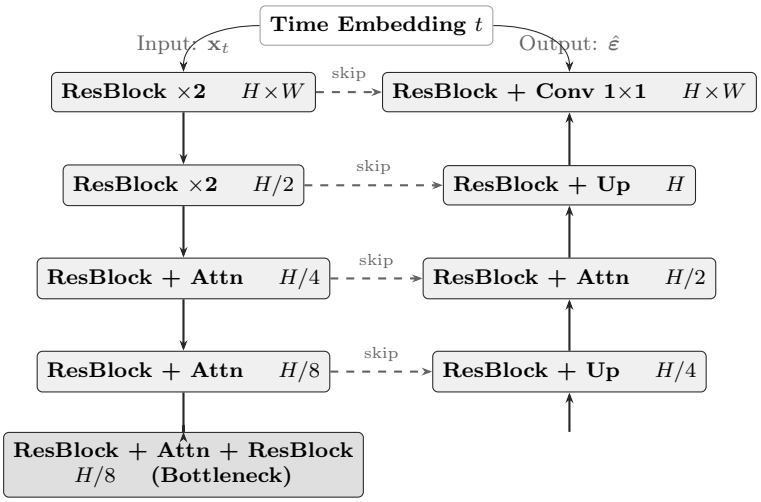

Figure 3: U-Net architecture used in DDPM. The encoder (left) progressively downsamples the feature maps. The decoder (right) upsamples back to the original resolution. Dashed arrows are skip connections that pass encoder features directly to the decoder, preserving spatial detail. The time embedding t is injected into every residual block.

where $\mathbf{e}_t[2i] = \sin(t/10000^{2i/d})$, $\mathbf{e}_t[2i+1] = \cos(t/10000^{2i/d})$. This tells the network *how noisy* its input is, so it can calibrate the strength of denoising.

Self-Attention. At coarser resolutions ($H/4$, $H/8$), the spatial feature map is flattened into a sequence of tokens and processed with multi-head self-attention:

$$\text{Attention}(Q, K, V) \;=\; \text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

This lets distant parts of the image communicate—e.g., ensuring a sky region stays consistent in colour across the full image.

Skip Connections. The encoder features at each resolution are *concatenated* to the corresponding decoder features. This allows the decoder to access fine-grained spatial information that would otherwise be lost during downsampling.

Table 2: U-Net Architecture Summary

Stage	**Resolution**	**Operation**	**Notes**
Encoder L1	$H \times W$	ResBlock ×2	Full resolution
Encoder L2	$H/2 \times W/2$	Downsample + ResBlock ×2	—
Encoder L3	$H/4 \times W/4$	ResBlock + Self-Attention	Attention begins here
Encoder L4	$H/8 \times W/8$	ResBlock + Self-Attention	Deepest encoder
Bottleneck	$H/8 \times W/8$	ResBlock + Attention + ResBlock	Global context
Decoder L4	$H/4 \times W/4$	ResBlock + Upsample + skip	—
Decoder L3	$H/2 \times W/2$	ResBlock + Attention + skip	—
Decoder L2	$H \times W$	ResBlock + Upsample + skip	—
Decoder L1	$H \times W$	ResBlock + Conv 1×1	Predicts $\hat{\boldsymbol{\epsilon}}$

5. Unified View and Practical Notes

5.1 How Everything Connects

All three frameworks—explicit score matching, DSM, and DDPM—are estimating the same object from different angles:

Table 3: Comparison of Score-Based Frameworks

Method	What the network learns	How to sample
Explicit Score Matching	$\nabla_{\mathbf{x}} \log p(\mathbf{x})$ directly	Langevin dynamics
DSM (Vincent 2011)	Gradient of noisy distribution; denoises $\tilde{\mathbf{x}}$ toward $\mathbf{x}_0$	Annealed Langevin
DDPM (Ho et al. 2020)	The noise $\boldsymbol{\varepsilon}$ added to $\mathbf{x}_0$ at each step t	Ancestral sampling (reverse Markov chain)

The key relationship: $\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) = -\sqrt{1-\bar{\alpha}_t} \cdot s_\theta(\mathbf{x}_t, t)$. The DDPM noise predictor and the DSM score network are the same thing, scaled differently.

5.2 Variance Schedules

The noise schedule $\{\beta_t\}$ controls how quickly the image is destroyed. Ho et al. used a **linear** schedule (β_t from 10^{-4} to 0.02). Nichol & Dhariwal (2021) proposed a **cosine** schedule, $\bar{\alpha}_t = \cos^2\left(\frac{t/T}{1.008} \cdot \frac{\pi}{2}\right)$, which decays more smoothly and avoids destroying fine image details too early.

5.3 Key Takeaways for Practitioners

Summary

- **DDPM training is simple:** just predict the noise with MSE. No adversarial game, no encoder–decoder balance to tune.
- **DDPM does not suffer from mode collapse** (a common GAN failure) because the forward process is fixed and covers all modes via noise.
- **Sampling is slow:** $T = 1000$ network forward passes per image. DDIM (Song et al., 2020) reduces this to $\sim$50 steps by making the reverse process deterministic.
- **The U-Net sees the timestep:** without the sinusoidal embedding, the network could not distinguish between a slightly noisy and a very noisy image and would fail to calibrate its denoising.
- **Score matching $\equiv$ DDPM training:** this unification (Song et al., 2021) opened the door to continuous-time formulations (stochastic differential equations) that generalize both frameworks.

References

[1] Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models.* NeurIPS 2020.

[2] Song, Y., & Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution.* NeurIPS 2019.

[3] Vincent, P. (2011). *A Connection Between Score Matching and Denoising Autoencoders.* Neural Computation.

[4] Song, J., Meng, C., & Ermon, S. (2020). *Denoising Diffusion Implicit Models.* ICLR 2021.

[5] Nichol, A., & Dhariwal, P. (2021). *Improved Denoising Diffusion Probabilistic Models.* ICML 2021.

[6] Song, Y. et al. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR 2021.